

VeriForge

แผนสร้างระบบตรวจสอบ AI-generated changes ให้ใช้งานจริง วัตถุประสงค์จริง และเริ่มด้วยงบต่ำ

เป้าหมาย

Change Assurance Platform สำหรับ repository ของเราและทีมเล็ก พร้อม FinTech sandbox ที่ใช้ชุดข้อมูลสาธารณะและการทำงานของระบบจริง เพื่อพิสูจน์วงจรตั้งแต่รับ issue จนถึง recovery

หลักการออกแบบ

AI เสนอและวิเคราะห์ · ระบบบังคับกฎจากหลักฐาน
มนุษย์อนุมัติตามความเสี่ยง · ทุก release ตรวจสอบย้อนกลับได้

สิ่งที่เอกสารนี้ครอบคลุม

1. ขอบเขตผลิตภัณฑ์และช่องว่างด้าน production
2. Architecture, identity, evidence และ financial controls
3. ข้อมูลสาธารณะ การทดสอบเสมือนจริง และตัวชี้วัด
4. บริการฟรี งบประมาณ และลำดับพัฒนา
5. เกณฑ์เปิด pilot และแหล่งอ้างอิง

จัดทำ 21 กันยายน 2026

เอกสารนี้เป็นแบบและเกณฑ์ตรวจสอบ ยังไม่ใช่ผล audit ของ implementation
ตัวเลขเป้าหมายและงบประมาณไม่ใช่ผลการทดลองหรือใบเสนอราคา

นิยามคำว่า “เหมือนจริง”

VeriForge ต้องรัน code, tests, scans, deployment และ recovery จริงใน environment ที่ควบคุมได้ โดยทุกขั้นตอนสร้างหลักฐานให้ตรวจสอบได้ การใช้ข้อมูลสาธารณะอย่างเดียวไม่เพียงพอที่จะพิสูจน์ว่าระบบ production ทำงานถูกต้อง

องค์ประกอบ	ข้อกำหนดรุ่นแรก
ผู้ใช้และ repository	ผู้ใช้ที่เชิญและ repos ที่อนุญาต มี auth และสิทธิ์จริง
Builder / Verifier	เรียก model จริงหรือ local model; บันทึกเวอร์ชัน ผลตอบกลับ เวลา และต้นทุน
Execution	รัน test/scanner จริง เก็บ exit code และ raw outputs
Release	Build image จริง ตรวจสอบ digest และ deploy ไป target จริง
Financial domain	Ledger และ constraints ทำงานจริง แต่ใช้เงินทดสอบ
Incident	ฉีด failure ใน sandbox แล้วเก็บ telemetry จากเหตุการณ์ที่เกิดจริง
Dashboard	อ่านจาก events/metrics ของระบบ; ไม่มีค่าความสำเร็จ hardcode

ขอบเขตที่ทำเสร็จได้ด้วยงบต่ำ

GitHub + Python/FastAPI + Docker + deployment target เดียว เริ่มจาก modular monolith สำหรับ control plane และ payment/fee/ledger อีกชุดเป็น target application

แยกป้ายกำกับข้อมูลให้ชัด

REAL_OPERATIONAL = ข้อมูลจากการใช้งานระบบของเรา

PUBLIC_BENCHMARK = ชุดโจทย์สาธารณะ

SYNTHETIC_REPLAY = ข้อมูลจำลองที่นำมารันซ้ำ

INJECTED_FAULT = ความผิดพลาดที่ฉีดเพื่อทดสอบ

การรันด้วยเงินจริงหรือข้อมูลลูกค้าลับไม่ได้เป็นเงื่อนไขของการพิสูจน์ engineering controls และยังไม่อยู่ในขอบเขต v1

สิ่งที่ต้องเพิ่มก่อน production

เรื่อง	ข้อกำหนด
Evidence provenance	ผูกผลกับ commit SHA, artifact digest, tool/test/policy versions และ identity [1]
Approval binding	อนุมัติ digest ใด deploy digest นั้น; code/policy เปลี่ยนต้องประเมินใหม่
Fail closed	Scanner timeout, evidence หาย, AI ผิด schema → INCONCLUSIVE และห้าม promote
Durable workflow	Persist state; มี lease, retry limit, deduplication, cancellation และ recovery งานค้าง
Authorization	ตรวจสอบสิทธิ์ทุก API/transition รวมถึงอ่าน evidence, approve, deploy, rollback
Sandbox	Runner แยกจาก control plane และ production credentials [2]
Data recovery	Backup DB/evidence นอกเครื่อง และทดสอบ restore
Operations	Alerts, retention, runbook, patching และเจ้าของ incident

AI สองตัวไม่ได้ทำให้การตรวจสอบอิสระโดยอัตโนมัติ

แยก identities, permissions และบริบท มี trusted acceptance tests ที่ Builder แก้มไม่ได้ การเปลี่ยน policy, CI หรือ trusted tests ต้องมีการอนุมัติเฉพาะ Verifier ขอรัน adversarial tests ได้ แต่แค่ release source หรือ deploy เองไม่ได้

Hard gates มาก่อน risk score

Financial invariant ผิดต้อง block แม้ risk score ต่ำ ข้อยกเว้นต้องระบุผู้อนุมัติ เหตุผล ขอบเขต วันหมดอายุ และข้อใดห้ามยกเว้น

Treat diff, logs และ incident memory เป็นข้อมูลที่ไม่เชื่อถือ ป้องกัน prompt injection ที่ชักนำให้ agent เปิดเผย secrets เพิ่มสิทธิ์ หรือปลอมผลตรวจ

กรณีหลัก: retry แล้วคิด fee ชำ

ธุรกรรมสำเร็จแล้วแต่ response timeout ทำให้ client retry ด้วย key เดิม ระบบต้องไม่สร้าง payment หรือ fee ชำ แม้หลาย requests เข้ามาพร้อมกัน

- กำหนด idempotency scope ตาม tenant + operation + key
- Key เดิมแต่ payload ต่างต้องถูกปฏิเสธ; กำหนด retention และผลตอบกลับชัดเจน [3]
- ใช้ unique constraints และ DB transaction ร่วมกับ concurrency tests
- เงินใช้ integer minor units หรือ decimal พร้อม rounding rule
- Ledger สมดุลแยก currency และ posting group
- Fee ถูกต้องตาม pricing rule version และสถานะธุรกรรม
- Refund/reversal เป็นรายการใหม่ที่อ้างอิงรายการเดิม
- External provider ต้องมี idempotency และ reconciliation ของตัวเอง

อย่าใช้เพียง $SUM(fee_events) \leq expected_fee$

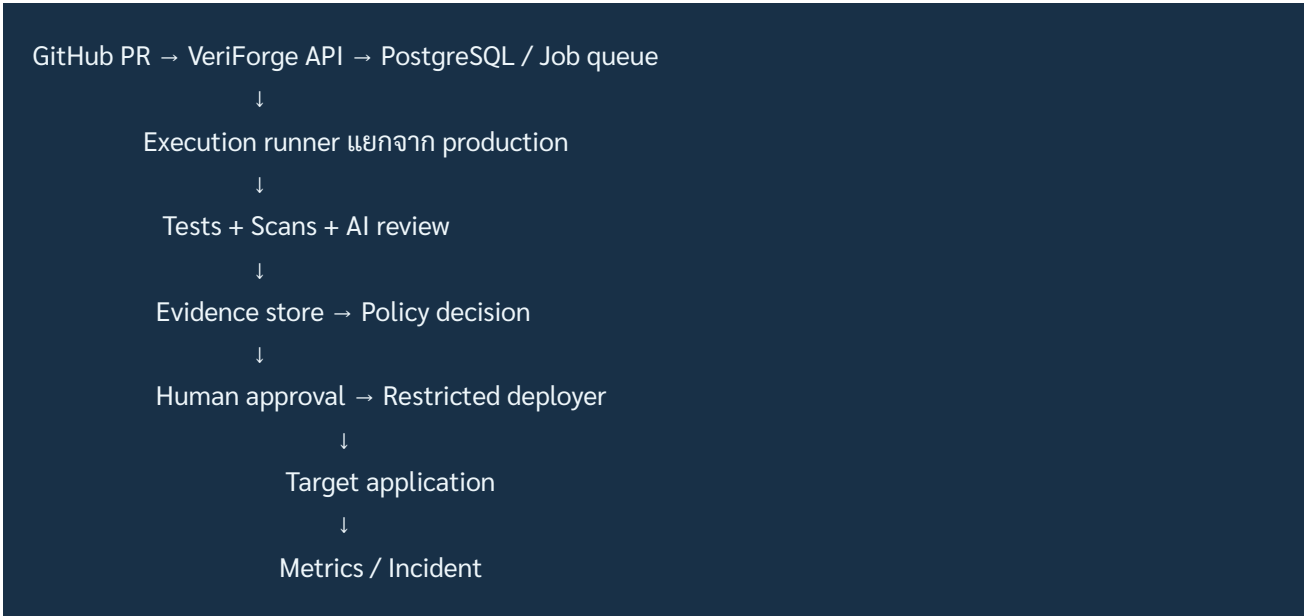
กฎนี้ยังผ่านได้เมื่อไม่เก็บ fee เลย หรือมีรายการผิดที่หักล้างกัน ต้องตรวจจำนวน posting ยอด reference และ lifecycle state ด้วย

Fault scenario	สิ่งที่ต้องพิสูจน์
Timeout หลัง commit	Retry คืนผลตาม contract โดยไม่สร้าง effect ชำ
Concurrent requests	มี financial effect ที่ถูกต้องเพียงครั้งเดียว
Worker crash / event replay	Recovery และ deduplication ทำงาน
Cross-tenant / rounding edge	แยกสิทธิ์และยอดเงินถูกต้อง

Rollback ไม่ย้อนเงิน: แยก application rollback, database compatibility/forward fix และ financial compensation หากพบยอดชำ อาจต้องหยุด write path ก่อน recovery ตาม runbook

Intentional bugs ต้องอยู่ใน fixtures, branches หรือ sandbox ที่แยกชัด ไม่เปิด vulnerable endpoints ต่อสาธารณะ

ระบบเล็กที่แยกอำนาจชัดเจน



ส่วน	แนวทางเริ่มต้น
Frontend	React + Vite + TypeScript หรือ Next.js static export
Control API / jobs	FastAPI + Pydantic + PostgreSQL job table
Orchestration / policy	Persistent state machine + versioned Python rules; เพิ่ม OPA ภายหลัง
Verification	pytest, financial tests, Gitleaks, Semgrep, Trivy
Execution / release	GitHub-hosted runner; Docker Compose + restricted deploy service
Evidence / monitoring	DB metadata + object storage; structured logs, metrics, OTel

ยังไม่ต้องมี Redis, Kafka, Kubernetes หรือ vector DB ใช้ PostgreSQL ค้น incidents ตาม service/failure type ก่อน VPS เล็กรัน API/DB/control worker ส่วน LLM และ scans แยกออกไป

กำหนด resource/time/network limits ให้ runner ห้ามมอบ production secrets หรือ privileged Docker access แก่งานที่ไม่เชื่อถือ Public demo ควรรับเฉพาะ repository ที่อนุญาต

เลือกข้อมูลให้ตรงกับสิ่งที่วัด

แหล่ง	ใช้ทำอะไร	ข้อจำกัด
SWE-bench / Verified [10]	โจทย์แก้ software issues จาก repos จริง; Verified มี 500 instances	วัด repair capability ไม่ครอบคลุม finance, deploy หรือ incident ทั้งวงจร; เสี่ยง benchmark contamination
PaySim [11]	ใช้รูปแบบ transactions และ amounts เป็นฐาน workload	เป็น synthetic data ไม่ใช่รายการธนาคารจริง; fraud label ไม่ใช่ label ของ software defect
Stripe sandbox [12]	ทดสอบ integration, payment lifecycle, decline/refund และ webhook handling	เงินไม่เคลื่อนไหวจริง; ไม่ใช่ sandbox ของ provider ทำ load test
GitHub / CI ของเรา	Issues, diffs, tests, build artifacts และ release events จริง	ต้องมี consent/scope และไม่ส่ง secrets ไป model
Runtime ของเรา	HTTP latency, DB locks, errors, ledger events และ recovery	ผลสะท้อน hardware/workload ที่ใช้ ต้องรายงาน เงื่อนไข

วิธีใช้ PaySim โดยไม่บิดความหมาย

สร้าง adapter แปลงเป็น API requests พร้อม synthetic accounts และ mapping ของ operation types ระบุแถวที่ตัดทิ้งและเหตุผล ใช้ distribution เป็นฐาน workload แต่ไม่อ้างว่าได้จำลอง settlement หรือบัญชีธนาคารจริงครบถ้วน

Dataset manifest ทุกชุด

บันทึก source URL, version/checksum, license/ข้อกำหนด, วันที่ดาวน์โหลด, schema, transform version, random seed, จำนวนแถว และ provenance type ตรวจสอบสิทธิ์ของ dataset แยกจาก license ของ simulator ก่อนแจกจ่าย

ข้อมูลจริงด้าน engineering ที่สำคัญที่สุดจะเกิดจากการรันระบบของเราเอง: commit → test execution → deployment → trace → incident → recovery ต้องเชื่อมด้วย change_id และ run_id

วัดผลให้ทำซ้ำและตรวจสอบได้

ชุดทดลองเริ่มต้นที่เสนอ — ยังไม่ใช่ผลทดลอง

เริ่มด้วย good/bad changes อย่างละ 20 เคส ครอบคลุม idempotency, authorization, rounding, concurrency และ migration แยก development/evaluation ตาม bug family หรือเวลา ไม่แยกสำเนา bug เดียวกันข้ามชุด

เปรียบเทียบ 3 วิธีบนชุดเดียวกัน: deterministic-only, deterministic + Verifier AI และวงจร Builder + Verifier จำกัดจำนวนรอบและงบเท่ากัน Label ความถูกต้องมาจาก specification/trusted tests และการตรวจของมนุษย์ ไม่ใช่ AI ตัวที่ถูกประเมินตัดสินเอง

Metric	นิยามที่ต้องบันทึก
Defect recall	Bad changes ที่ block ถูกต้อง / bad changes ทั้งหมด
False block rate	Good changes ที่ block ผิด / good changes ทั้งหมด
Inconclusive rate	ผลตัดสินไม่ได้ / runs ทั้งหมด; แยกจาก PASS/BLOCK
Bug escape rate	Bad changes ที่ได้รับอนุญาตผ่าน / bad changes ทั้งหมด
Latency / error rate	p50/p95/p99 และ failed requests / total ตาม window เดียวกัน
MTTD / MTTR	Fault start → detection; impact start → verified recovery
Cost per change	Model usage + runner/storage allocation; local usage ระบุต้นทุนโดยประมาณ

Performance และ fault tests

เริ่ม workload 1, 5, 20 requests/second บนระบบของเรา พร้อม warm-up และ steady-state; เพิ่มจนพบขีดจำกัด บันทึก achieved throughput, concurrency, CPU/RAM, DB version และ image digest ใช้ histogram สำหรับ latency [13]

ทดสอบ timeout, process crash, DB lock และ event replay แยกกันก่อนทำ combined faults ทำซ้ำอย่างน้อย 3 รอบ รายงาน sample size, median/range และ uncertainty; ห้ามอ้างความแม่นยำสูงจากจำนวนเคสน้อย

Canary traffic ต้องมี synthetic traffic และจำนวนตัวอย่างขั้นต่ำ ข้อมูลไม่พอให้ INCONCLUSIVE; ไม่ยิง load test ไปยัง payment provider [12]

ใช้ฟรีก่อน จ่ายเมื่อจำเป็น

รูปแบบ	ความเหมาะสม
เครื่องที่มีอยู่ + frontend/CI ฟรี	Cloud อาจเป็นศูนย์; มีค่าไฟและเน็ต งานหยุดเมื่อปิดเครื่อง
Free cloud ทั้งชุด	Pilot ที่ยอมรับ quota และ resource pause/reclaim ได้
VPS + CI + backup แยก	กรอบวางแผน €10–20/เดือน ไม่รวม AI API/domain; ต้องวัด resource จริง

- **Cloudflare Pages:** static requests ฟรี; Functions มี quota แยก [4]
- **GitHub Actions:** standard hosted runners ของ public repos ฟรี; private บน GitHub Free มี 2,000 นาที/เดือน และ artifact storage 500 MB [5]
- **Supabase Free:** DB 500 MB; พัก project หลังไม่มี activity หนึ่งสัปดาห์ [6]
- **Oracle Always Free:** idle compute อาจถูกเรียกคืน [7]
- **R2:** ติดตามทั้ง storage และ operation costs [8]
- **VPS ตัวอย่าง:** Hetzner CX23 ยุโรป €5.49/เดือน ก่อน VAT และ IPv4 ตามตารางที่ตรวจ ไม่รวม backup [9]

ลดต้นทุน AI โดยยังตรวจสอบจริง

รัน checks ราคาถูกก่อนเรียก AI กำหนด token limit, timeout, concurrency, jobs/day และรอบแก้ไขเริ่มต้นไม่เกิน 2 รอบต่อ change Cache ผูกกับ commit, policy, evidence และ model configuration

Budget cap ต้องหยุด AI jobs ใหม่ได้จริง หาก AI ใช้ไม่ได้ให้แสดง review ไม่ครบ และห้าม promote เมื่อ policy บังคับ review

Local model ลด API fees แต่มีต้นทุนเครื่อง/ไฟฟ้าและต้องทดสอบคุณภาพ VPS เครื่องเดียวมี single point of failure ให้แยก backup ออกไปและทดสอบ restore

ราคา/quota ตรวจสอบเอกสารวันที่ 21 ก.ย. 2026 อาจเปลี่ยนได้ ต้องตรวจอีกครั้งก่อนสมัคร GitHub Actions ใช้สำหรับ build/test ของ repository ไม่ใช่เป็น backend agent ตลอดเวลา

แผนพัฒนาและเกณฑ์เปิด pilot

ระยะ	สิ่งส่งมอบ
1 · Foundation	Auth, roles, threat model, repo allowlist, audit, data manifest
2 · Financial testbed	Payment/fee/ledger; duplicate-fee reproducer และ trusted tests
3 · Assurance	รับ commit → รัน tests/scans จริง → evidence → policy decision
4 · Independent AI	Verifier findings/tests; Builder patch ผ่านวงจรเดิม
5 · Release	Artifact-bound approval, staging, deploy, health checks, rollback
6 · Evaluation / pilot	Replay datasets, benchmark, fault tests, backup/restore, cost controls

ต้องพิสูจน์ผ่านก่อนเปิดใช้

1. Bad patch ถูก block; good patch ผ่าน replay/concurrency tests
2. Builder อนุมัติหรือ deploy งานตัวเองไม่ได้
3. แก้ commit หลัง approval แล้ว approval เดิมใช้ไม่ได้
4. Evidence ปลอมหรือคนละ artifact ถูกปฏิเสธ
5. Scanner timeout / worker crash ไม่ false pass หรือ deploy ช้า
6. Prompt injection ใน diff/log ไม่เพิ่มสิทธิ์ agent
7. Runtime regression หยุด promotion หรือ recovery ตาม policy ได้
8. Restore DB และ evidence ลง environment ใหม่ได้
9. ถึง budget cap แล้วหยุดงานตามข้อกำหนด
10. ทุกตัวเลขบน dashboard ย้อนกลับถึง raw evidence ได้

งานแรก

ทำ duplicate-fee reproducer → trusted regression test → evidence bundle → deterministic BLOCK ให้ครบก่อนต่อ AI และ deployment

ออกจากแต่ละระยะเมื่อผ่าน acceptance criteria เป้าหมายนี้คือระบบตรวจ change ที่ใช้งานจริงในขอบเขตจำกัด ยังไม่ใช่ certification หรือการรับรองพร้อมรับเงินจริง

แหล่งอ้างอิง

1. SLSA — Provenance · การตรวจสอบแหล่งที่มาของ artifacts
2. GitHub Actions — Secure use reference · Runner และ workflow security
3. Stripe — Idempotent requests · Contract สำหรับ safe retries
4. Cloudflare Pages — Pricing · Static assets และ Functions
5. GitHub Actions — Billing · Minutes และ artifact storage
6. Supabase — Pricing · Free plan และ pausing
7. Oracle — Always Free resources · Free compute restrictions
8. Cloudflare R2 — Pricing · Storage และ operations
9. Hetzner — Price adjustment · ราคาอ้างอิง cloud servers
10. SWE-bench — Official benchmark · โจทย์ software engineering
11. PaySim — Project repository · Mobile-money simulator และลิงก์ dataset
12. Stripe — Testing · Payment sandbox และข้อจำกัด load testing
13. Prometheus — Histograms and summaries · การวัด latency distributions

หลักฐานที่ควรส่งมอบคู่กับโปรเจกต์

Architecture / threat model · Dataset manifests · Reproducible run instructions · Raw evidence
· Evaluation report · Cost report · Restore/rollback report · Known limitations

ลิงก์ใน PDF คลิกได้ ข้อเสนอแนะด้าน architecture และชุดทดลองเป็นการออกแบบสำหรับ VeriForge ไม่ใช่ข้ออ้างว่าบริการอ้างอิงรับรองระบบนี้